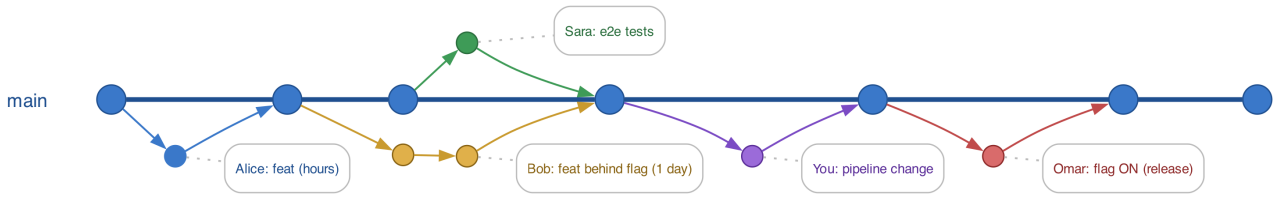
Luul Team
Trunk-Based Project

Developers, QA, lead engineers and you (the platform engineer): roles, permissions, 10 real problems with tested solutions



Everybody merges small changes into one trunk, every day.

Project folder: `~/wezva-ai-driven-bootcamp/git-github/luul-team-project`

Green boxes = what to say in the interview. Blue boxes = the guardrail you build as a platform engineer.

Contents

Part A: the platform view

1. The team, roles and permissions

2. How one change flows through the team

3. Environments, approvals and releases

4. Your job: building the guardrails

Part B: the developer's view

5. The app explained

6. Developer lab: build a feature yourself

7. Two points of view: the same change

8. When it goes wrong: both sides

Part C: practise

9. The team simulator

Scenarios 1-10: real team problems and how to solve them

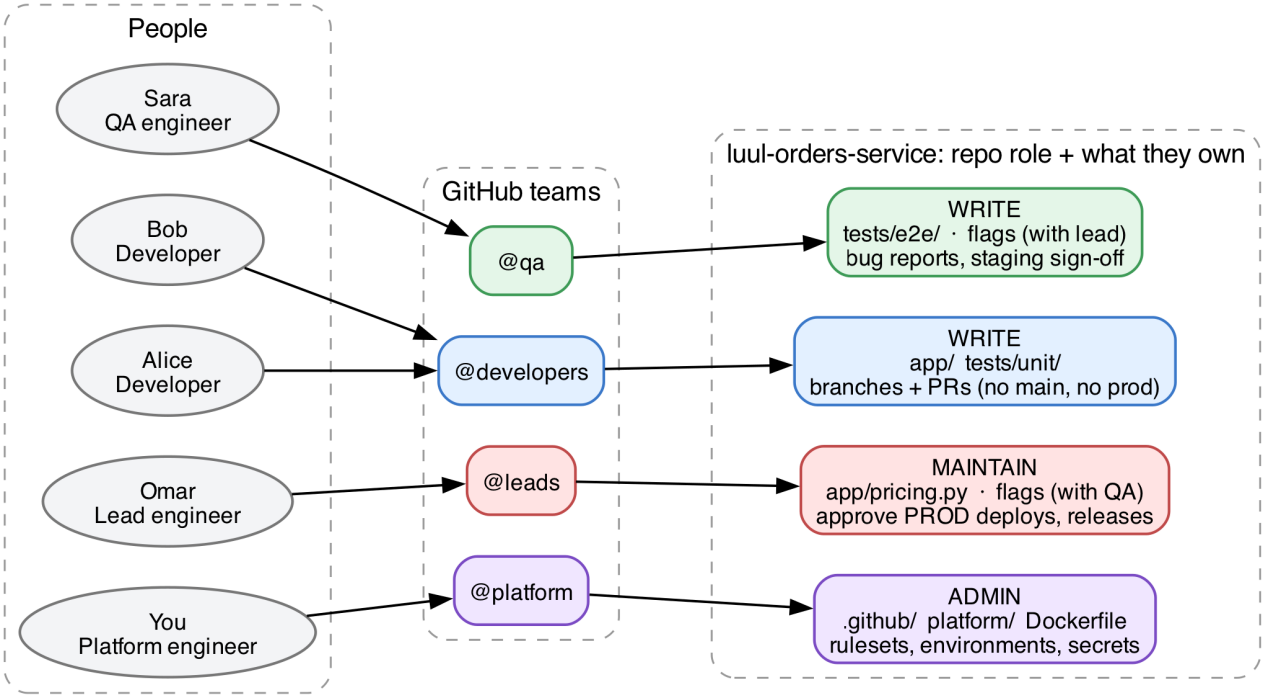
10. Practise on real GitHub

11. Put it on your CV

12. Interview questions and answers (54)

13. Cheat sheet

1. The team, roles and permissions



People belong to GitHub teams. Teams get a repo role and own paths through CODEOWNERS.

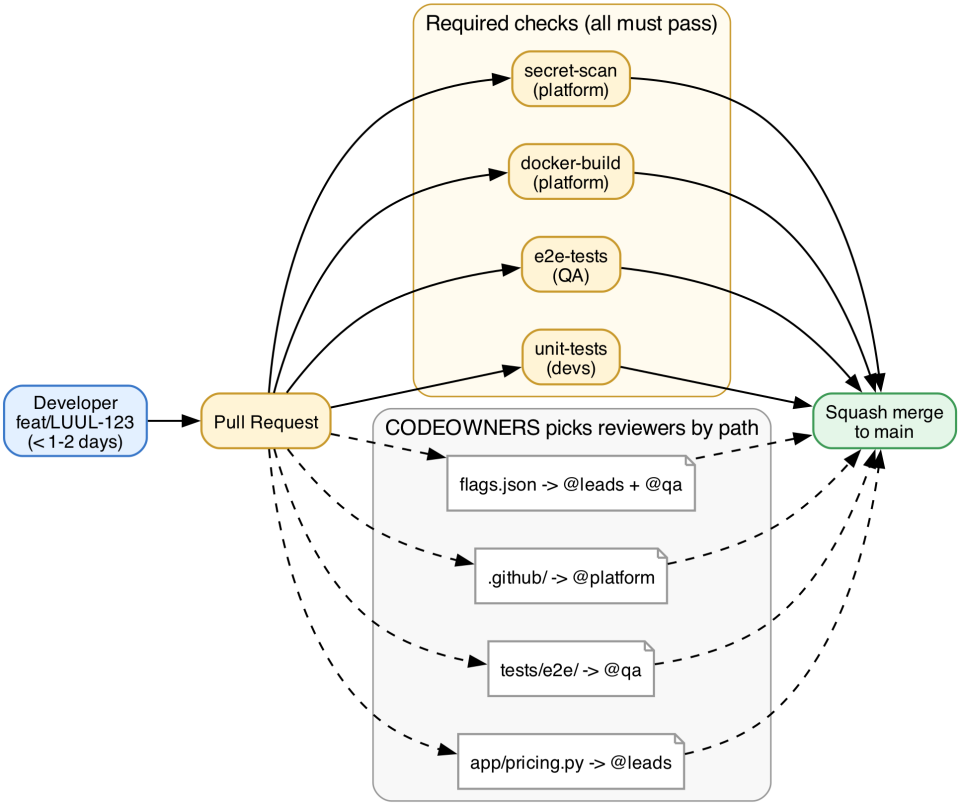
GitHub repo roles, from least to most power: **Read, Triage, Write, Maintain, Admin**. Each team gets the **least** power it needs (*principle of least privilege*).

Action	Dev	QA	Lead	Platform
Create branches, push branches, open PRs	yes	yes	yes	yes
Push directly to main	no	no	no	no (only audited break-glass via PR)
Approve app/pricing.py (money logic)	no	no	yes	no
Approve tests/e2e/	no	yes	no	no
Approve turning a feature flag ON	no	yes + lead	yes + QA	no
Approve .github/, Dockerfile, platform/	no	no	no	yes
Approve a PROD deployment	no	no	yes	yes
Releases / tags	no	no	yes	yes
Repo settings, rulesets, environments, secrets	no	no	no	yes
Onboard / offboard people	no	no	no	yes (org owner)

Say this in the interview

We map people to GitHub teams with least privilege: developers and QA have write, leads have maintain, and platform has admin. CODEOWNERS makes sure the right team reviews the right files: leads for pricing, QA for e2e tests, platform for pipelines. Prod deployments need a lead or platform approval through GitHub environments.

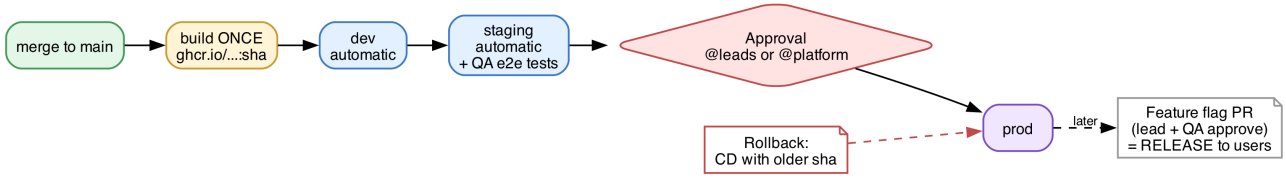
2. How one change flows through the team



Every PR must pass 4 required checks AND get approval from the code owners of the files it touches.

- **Developer**: branch from the latest main (`feat/LUUL-123-x`), small change, unfinished work behind a flag, open a PR the same day.
- **CI**: `unit-tests` (devs), `e2e-tests` (QA), `docker-build` and `secret-scan` (platform). All required.
- **CODEOWNERS**: GitHub automatically requests the right reviewers.
- **Squash merge**: one clean commit per PR on main. The branch is deleted automatically.
- **Repeat daily.** Everyone rebases on main every morning: `git pull --rebase origin main`.

3. Environments, approvals and releases



Build once, promote the same image. Prod waits for a human. The feature is released later with a flag PR.

Environment	Who / what deploys	Gate
dev	Automatic on every merge to main	none
staging	Automatic after dev	only main may deploy; QA e2e tests run against it
prod	Same image after staging	required reviewers: @leads or @platform; only main
rollback	`gh workflow run CD -f sha=$(git rev-parse <good-sha>)` (the FULL commit ID)	same approvals; uses the code, tests and settings of that commit

Deploy is not release. Code reaches prod with the flag OFF. QA tests it in staging with `FLAG_DISCOUNTS_V2=true`. Then a tiny PR flips `config/flags.json`, and CODEOWNERS requires **lead + QA** approval. That PR *is* the release.

Releases with versions: a tag `v1.2.0` on main runs `release.yml`. To patch an old version, use a release branch + cherry-pick (Scenario 9).

4. Your job: building the guardrails

As the platform engineer you build the rules once, so every team can move fast safely. Two ways, same result:

```
# a) GitHub CLI script
./platform/setup-org.sh <practice-org> luul-orders-service

# b) Terraform: "GitHub as code" (reviewed in PRs, auditable, repeatable)
cd platform/terraform && export GITHUB_TOKEN=$(gh auth token)
terraform init && terraform plan && terraform apply
```

Guardrail	Setting	Why
Ruleset protect-trunk	PR required, 1 approval, code-owner review, last-push approval, conversations resolved	Nobody changes main without review
	No force-push, no deletion, linear history	History is safe and readable
	Required checks + branch up to date	Main stays green (Scenario 3)
	Bypass: org admins, pull_request mode only	Break-glass, audited, never direct pushes
CODEOWNERS	path -> team	The right expert reviews
Environments	staging/prod main-only, prod reviewers	Segregation of duties
Merge settings	squash only, auto-delete branches, auto-merge	Clean trunk, no stale branches
Security	secret scanning + push protection, gitleaks job, pre-commit hook, Dependabot	Scenarios 4 and 5

A piece of the Terraform

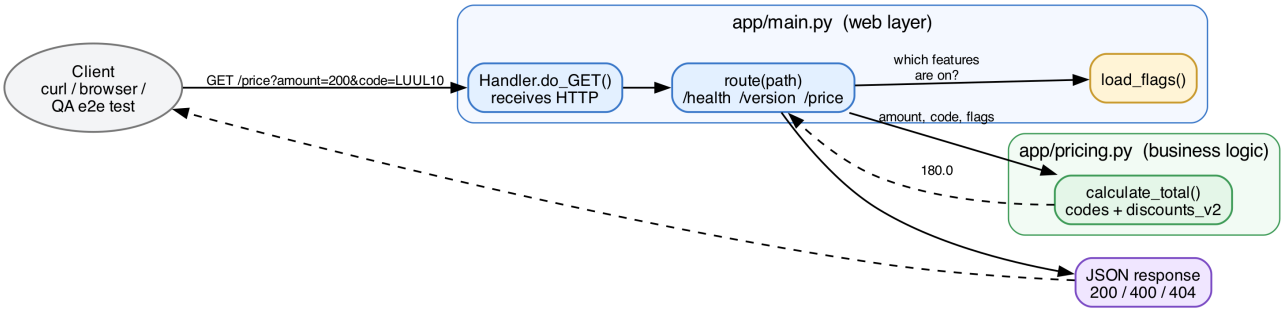
```
resource "github_repository_ruleset" "protect_trunk" {
  name        = "protect-trunk"
  repository  = var.repo
  target      = "branch"
  enforcement = "active"
  conditions { ref_name { include = ["~DEFAULT_BRANCH"], exclude = [] } }
  rules {
    deletion                = true
    non_fast_forward        = true
    required_linear_history = true
    pull_request {
      required_approving_review_count = 1
      require_code_owner_review       = true
    }
    required_status_checks { ... unit-tests, e2e-tests, docker-build, secret-scan }
  }
}
```

> **Safety**
>
> Practise in your OWN practice organization. The script and the Terraform both refuse to run against the real Luul-Solutions org. Changing company settings needs your lead's approval.

5. The app explained

As a platform engineer you serve developers, so you must understand **their** work.
`luul-orders-service` is a small web API: send a price and a code, get JSON back.

```
curl "localhost:8080/price?amount=200&code=LUUL10"
{"amount": 200.0, "code": "LUUL10", "total": 180.0}
```

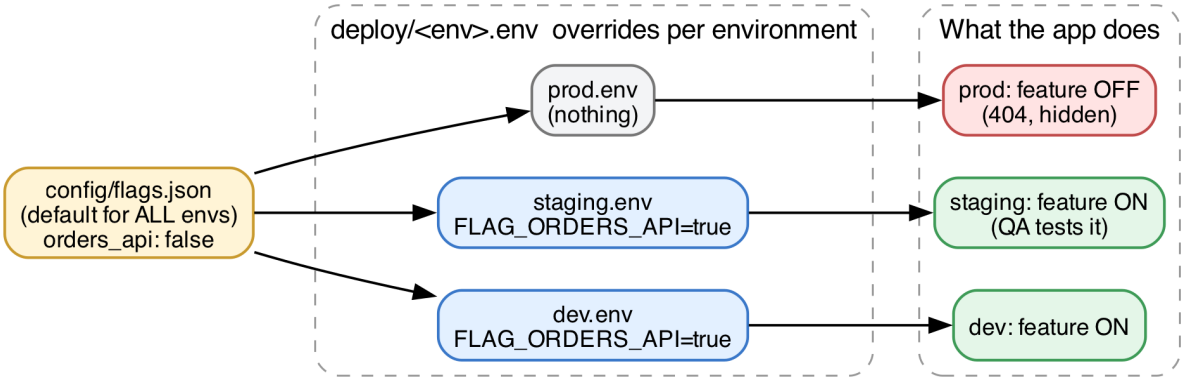


One request: the web layer (main.py) asks which flags are on, calls the business logic (pricing.py), returns JSON.

File	What it does	Who changes it
`app/pricing.py`	**Business logic.** DISCOUNT_CODES maps codes to percentages. calculate_total() applies them. discounts_v2 adds 5% on orders of 500 or more.	Developers (lead approves)
`app/main.py`	**Web layer.** route() maps URLs to code. Bad input gives 400, never a crash (500).	Developers
`config/flags.json`	**Feature switches**, the default for all environments	Lead + QA approve
`deploy/<env>.env`	**Per-environment settings.** FLAG_X=true overrides flags.json in that env only	Platform + leads
`tests/unit/`	Call functions directly. Milliseconds, no server	Developers
`tests/e2e/`	Start the real server and call it like a customer. With BASE_URL they test staging	QA
`Dockerfile`	Python 3.12, copy app/ + config/, non-root user, /health check	Platform
`Makefile`	make test / run / build: the same commands locally and in CI	Everyone

Three ideas every developer (and you) must know

- **Status codes:** 200 OK, 400 bad input from the client, 404 not found or hidden, 500 our code crashed. A 500 is always a bug, and alerts are built on these numbers.
- **/health** is for machines (Docker, Kubernetes, load balancers). **/version** shows which commit is running: the first thing to check in an incident.
- **Configuration is not code.** The same image runs everywhere, and only settings change per environment.



Where a flag's value comes from: flags.json is the default, and deploy/.env overrides it per environment.

6. Developer lab: build a feature yourself

Ticket LUUL-400: *"As a customer, I want an order summary."* `GET /orders/summary?items=100,50&code=LUUL10` returns subtotal 150 and total 135. Not ready for customers yet, so it goes behind the flag **orders_api**. Reference solution: `docs/dev-lab-solution/`. Full text: `docs/DEVELOPER-VIEW.md`. **Every step below was run and tested.**

Steps 1-2: You are Alice (developer). Branch and add the flag, OFF

```
cd $W/alice-dev
git switch main && git pull
git switch -c feat/LUUL-400-order-summary
# config/flags.json ->  { "discounts_v2": false, "orders_api": false }
git commit -am "feat(LUUL-400): add orders_api flag (off)"   # safe to merge right away
```

Steps 3-4: Test first, then code (test-driven development)

```
cp <solution>/tests/unit/test_orders.py tests/unit/
make test-unit        # FAILS: No module named 'app.orders'   <- expected
# write app/orders.py: summarize(prices, code, flags) -> items, subtotal, total
# app/main.py: import summarize + an "/orders/summary" block
#              (it returns 404 when the flag is OFF)
make test-unit        # Ran 13 tests ... OK
git add -A            # -A includes the NEW file app/orders.py
git commit -m "feat(LUUL-400): order summary endpoint behind orders_api flag"
```

Step 5: Try it like a user, then push

```
make run
curl "localhost:8080/orders/summary?items=100,50&code=LUUL10"
# {"error": "not found"}   <- hidden, as planned
FLAG_ORDERS_API=true make run
curl "localhost:8080/orders/summary?items=100,50&code=LUUL10"
# {"items": 2, "subtotal": 150.0, "code": "LUUL10", "total": 135.0}
git push -u origin feat/LUUL-400-order-summary   # GitHub: gh pr create
```

Step 6: You are Sara (QA). Add e2e tests on the same branch

```
cd $W/sara-qa && git fetch && git switch feat/LUUL-400-order-summary
cp <solution>/tests/e2e/test_orders_api.py tests/e2e/
make test-e2e          # Ran 6 tests ... OK
git add -A && git commit -m "test(LUUL-400): e2e for order summary" && git push
```

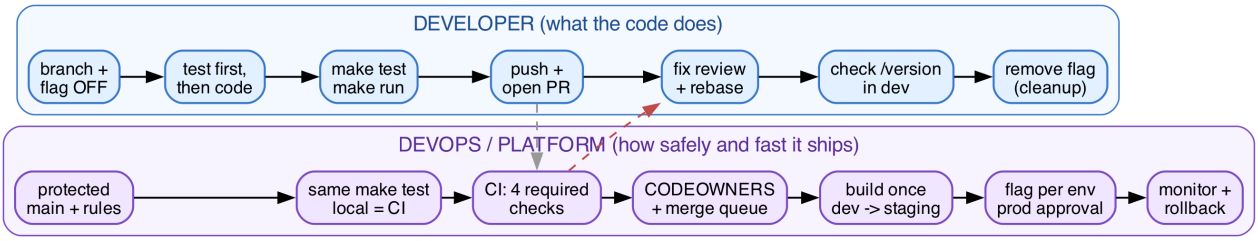
Step 7: You are you (platform). Flag ON in dev and staging only

```
cd $W/you-platform && git switch main && git pull
git switch -c chore/LUUL-401-orders-flag-envs
echo "FLAG_ORDERS_API=true" >> deploy/dev.env
echo "FLAG_ORDERS_API=true" >> deploy/staging.env    # CD runs QA's e2e against staging
git commit -am "chore(LUUL-401): orders_api ON in dev + staging"
git push origin HEAD:main         # GitHub: PR, platform + lead approve
```

Steps 8-9: Alice merges. Later, lead + QA release it

```
cd $W/alice-dev
git pull --rebase && git fetch origin && git rebase origin/main
make test                          # unit 13 OK + e2e 6 OK
git push --force-with-lease
git push origin HEAD:main          # GitHub: squash merge
# Release (later): PR "orders_api": true in flags.json -> lead + QA
# Cleanup (weeks later): remove the flag and the if-check
```

7. Two points of view: the same change



Developers own WHAT the code does. DevOps owns HOW SAFELY AND QUICKLY it reaches users. A red check sends the developer back to fix and rebase. They meet in the PR and the pipeline.

Step	Developer does / sees	DevOps / platform does / sees
Start	Branch from latest main, read the ticket	Protected main and a fast CI, so small PRs are cheap
Flag	Adds orders_api: false	Owns the flag mechanism and deploy/<env>.env
Code + tests	Test fails, write code, test passes. make test	make test locally = the same command in CI
PR	Opens a PR, waits for checks	4 required checks. Watches duration and flakiness
Review	Gets review from CODEOWNERS	Designed CODEOWNERS per path
QA	Sara adds e2e tests	e2e is required AND runs against staging
Settings	Doesn't touch environments	Flag ON in dev + staging via a reviewed PR
Merge	Rebase, green, squash merge	Merge queue / up-to-date rule keeps main green
Deploy	Checks /version in dev	Build once :sha, dev, staging (+e2e), prod approval
Release	Lead + QA approve the flag-on PR	Monitors errors and latency. Rollback ready
Cleanup	Removes the flag code	Removes the flag from env files

Say this in the interview

I see my job as making the safe path the easy path for developers. The same make test runs on their laptop and in CI, pipelines are fast, settings are per environment, and when something fails I debug it with them. Developers own what the code does. I own how safely and quickly it reaches users.

8. When it goes wrong: both sides

Case 1: QA's tests pass locally but fail in staging (tested)

Developer sees	Real cause
CD job deploy-staging fails: AssertionError: 404 != 200 on /orders/summary	Code is fine. The **flag is OFF in staging**: deploy/staging.env doesn't have it

```
cat deploy/staging.env            # FLAG_ORDERS_API=true there?
curl https://staging.../version    # is the right commit deployed?
```

Fix / lesson: a PR adding the flag to staging.env. *Same code, different config:* compare configuration first.

Case 2: It works on my laptop but CI fails (tested)

Developer sees	Real cause
CI unit-tests: ModuleNotFoundError: No module named 'app.helpers'. Green on the laptop	The new file was never committed. **git commit -am** skips NEW files, and CI starts from a clean clone

```
git status --short    # ?? app/helpers.py   <- untracked = never committed
git add app/helpers.py && git commit -m "..." && git push
```

Fix / lesson: CI is a clean room. Same symptom, other causes: a local env var (env | grep FLAG_), a different Python version, a package installed only locally.

Case 3: It works locally but not in Docker

Developer sees	Real cause
Container starts, then the health check fails, or a file is missing	Folder not in the Dockerfile COPY lines, excluded by .dockerignore, listening on 127.0.0.1 not 0.0.0.0, or a missing env var

```
docker logs <container>
docker run --rm -it <image> ls -R /srv        # is the file INSIDE the image?
docker run --rm -it <image> env | grep FLAG   # which settings does it see?
```

Fix / lesson: the image only contains what the Dockerfile copies. Check inside the image, not on the laptop.

Case 4: My PR can't be merged

Message on the PR	Meaning	Developer fix
Review required	A CODEOWNER hasn't approved	Ask the team shown in the PR
Branch is out-of-date	The rule requires the latest main	Update branch, or rebase + --force-with-lease
Some checks were not successful	A required check failed	Open Details on the red check, read the log
Changes requested	Reviewer wants changes	Push fixes, resolve conversations

DevOps: explain the rule. Don't bypass it. If a rule only slows people down, change it with a PR.

Case 5: The pipeline takes 25 minutes

Developers lose focus and make **bigger** PRs to wait less often, which breaks trunk-based development. DevOps measures first (`gh run view <id>` shows time per job), then caches, runs jobs in parallel, uses path filters, splits slow tests, and cancels outdated runs. Goal: PR checks in under 10 minutes.

Case 6 and 7: Port in use / bug only in prod

```
lsof -i :8080 ; kill <PID>      # or: PORT=8081 make run

curl https://prod.../version                        # which commit is live?
diff deploy/staging.env deploy/prod.env             # what is different?
set -a; . deploy/prod.env; set +a; make run         # reproduce with prod settings
```

9. The team simulator

The simulator gives every person their own folder and Git identity, so you can see the problems happen between real people. You switch between their folders and fix the problems with real commands.

```
cd ~/wezva-ai-driven-bootcamp/git-github/luul-team-project
./sim/create-team.sh --local        # once

# creates ../luul-team-workspace/
#   github-origin.git   <- acts as "GitHub"
#   you-platform/       Abdihakim (Platform)
#   alice-dev/          Alice (Developer)
#   bob-dev/            Bob (Developer)
#   sara-qa/            Sara (QA)
#   omar-lead/          Omar (Lead)

./sim/scenarios/s01-commit-on-main.sh   # creates the problem (resets everything first)
W=~/wezva-ai-driven-bootcamp/git-github/luul-team-workspace
cd $W/bob-dev                           # become Bob and fix it
./sim/reset.sh                          # back to the start any time
```

#	Scenario	Main skill
1	Bob committed on main	branch pointers, reset
2	Merge conflict between two developers	rebase, resolve, --force-with-lease
3	main is RED after two green PRs	bisect, revert, merge queue
4	A secret was pushed	rotate, rewrite branch, push protection
5	12 MB database dump in a commit	rm --cached, amend, .gitignore, LFS
6	QA lost a day of work	reflog
7	2-week big-bang branch	split with cherry-pick, flags
8	Bug in staging: which commit?	bisect run, regression test
9	Hotfix for a version in production	release branch, cherry-pick -x, tags
10	Commits with the wrong author	rebase --exec, identity, signing

Every solution in this guide was tested end to end on the simulator.

Scenario 1: Bob committed on main by mistake

Who	Folder	Script
Bob (Developer)	`$W/bob-dev`	`./sim/scenarios/s01-*.sh`

Problem. Bob forgot to branch and made 2 commits on local main. On GitHub the push is rejected: *GH013: Repository rule violations found... Changes must be made through a pull request.*

Solve it

```
git status -sb                              # main...origin/main [ahead 2]
git switch -c feat/LUUL-201-student20       # new branch keeps the 2 commits
git switch main
git reset --hard origin/main                # main matches GitHub again
git switch feat/LUUL-201-student20
make test
git push -u origin feat/LUUL-201-student20  # then open a PR
```

Prevent it (platform guardrail)

The ruleset rejects pushes to main. `make hooks` installs a pre-commit hook that blocks commits on main.

Say this in the interview

I move the commits to a feature branch, reset local main to origin/main, and open a PR. A branch is just a pointer, so nothing is lost.

Scenario 2: Merge conflict between two developers

Who	Folder	Script
Bob (Developer) + Omar (Lead)	`$W/bob-dev`	`./sim/scenarios/s02-*.sh`

Problem. Alice's PR (LUUL10 = 15%) is merged. Bob's PR changes LUUL10 to 12% **and** adds SUMMER20 on the same lines. Omar decides: keep 15%, keep SUMMER20.

Solve it

```
git fetch origin
git rebase origin/main          # CONFLICT: app/pricing.py, tests/unit/test_pricing.py

# app/pricing.py: keep BOTH decisions, delete the <<<<<<< ======= >>>>>>> lines
    "LUUL10": 15,   # 15% off
    "SUMMER20": 20, # summer campaign

git checkout --ours tests/unit/test_pricing.py   # rebase: ours = origin/main
git add app/pricing.py tests/unit/test_pricing.py
git rebase --continue
git commit --amend -m "feat(LUUL-211): add SUMMER20 code"
make test
git push --force-with-lease
```

Prevent it (platform guardrail)

Small PRs merged the same day, and everyone rebases on main every morning. Remember: in a **rebase**, 'ours' is the branch you rebase onto. In a **merge**, 'ours' is your branch.

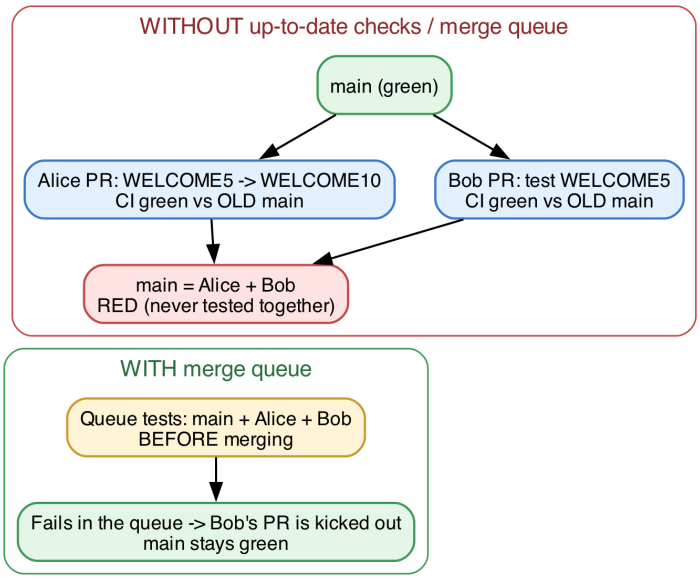
Say this in the interview

I rebase on main, resolve it with the other author or the lead when it's a business decision, run the tests, and push with --force-with-lease so I never overwrite someone else's work.

Scenario 3: main is RED after two green PRs

Who	Folder	Script
You (Platform) + Alice + Bob	`$W/you-platform`	`./sim/scenarios/s03-*.sh`

Problem. Alice replaced WELCOME5 with WELCOME10. Bob added a test for WELCOME5. Both PRs were green against the old main, and there was no textual conflict. Together they break main, so nobody can ship.



Why two green PRs can make a red main, and how a merge queue stops it.

Solve it

```
git pull
make test-unit                      # FAILS
git bisect start HEAD sim-start     # bad = now, good = last known good
git bisect run make test-unit       # -> "<sha> is the first bad commit"
git bisect reset
git switch -c revert/LUUL-221
git revert <sha>                    # new commit that undoes it
make test-unit                      # green
git push origin HEAD:main           # GitHub: PR + fast review
# then Bob fixes his test properly: WELCOME10 -> 180
```

Prevent it (platform guardrail)

Require branches to be up to date before merging (strict checks), and turn on a **merge queue**. `ci.yml` already runs on `merge_group`.

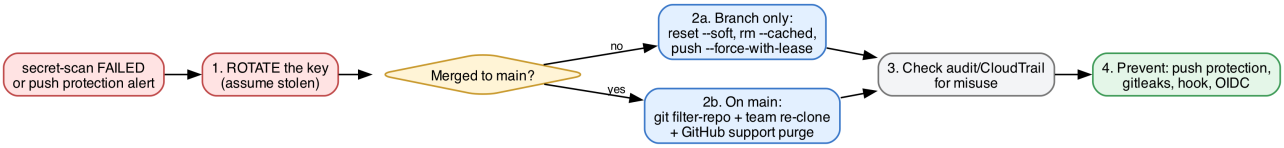
Say this in the interview

A broken trunk blocks everyone, so: revert first, fix forward second. Semantic conflicts are prevented with up-to-date checks and a merge queue that tests PRs together with the latest main.

Scenario 4: A secret was pushed

Who	Folder	Script
Bob (Developer) + You (Platform)	`$W/bob-dev`	`./sim/scenarios/s04-*.sh`

Problem. The secret-scan check failed. The first commit of Bob's pushed branch contains config/aws.env with an AWS key.



Secret leak response.

Solve it

```
# 1. ROTATE FIRST (assume it's stolen)
aws iam create-access-key --user-name luul-export
aws iam delete-access-key --user-name luul-export --access-key-id <OLD>

# 2. Remove it from the branch history
git log --oneline --stat origin/main..HEAD
git reset --soft origin/main         # un-commit the branch, keep changes staged
git rm --cached config/aws.env && rm config/aws.env
echo "*.env" >> .gitignore && git add .gitignore
git commit -m "feat(LUUL-230): export module (credentials from env)"
git push --force-with-lease

# Already on main? git filter-repo --path config/aws.env --invert-paths
#   + whole team re-clones + ask GitHub Support to purge
```

Prevent it (platform guardrail)

Push protection, the gitleaks job, the pre-commit hook, and **OIDC** so CI never needs static cloud keys.

Say this in the interview

Rotate first, clean history second. A key on GitHub for even a minute is compromised. Then I add push protection and gitleaks, and move CI to OIDC so there are no long-lived keys.

Scenario 5: A 12 MB database dump in a commit

Who	Folder	Script
Alice (Developer)	`$W/alice-dev`	`./sim/scenarios/s05-*.sh`

Problem. Alice's last commit (not pushed) contains data/prod-dump.sql together with her real code change. It's big, and it may contain customer data.

Solve it

```
git show --stat HEAD
git rm --cached data/prod-dump.sql
rm -rf data/
printf "data/\n*.sql\n" >> .gitignore && git add .gitignore
git commit --amend --no-edit       # fine: not pushed yet
git show --stat HEAD               # only code + .gitignore
make test
```

Prevent it (platform guardrail)

The pre-commit hook blocks files over 5 MB. Big files you really need go in Git LFS or S3. Production data never goes on laptops.

Say this in the interview

Since it wasn't pushed, I amended the commit. With possible customer data, I also treat it as a data-protection incident.

Scenario 6: QA lost a day of work

Who	Folder	Script
Sara (QA)	`$W/sara-qa`	`./sim/scenarios/s06-*.sh`

Problem. Sara ran `git reset --hard origin/main` and her 2 new e2e test commits disappeared.

Solve it

```
git reflog
# 9d281fc HEAD@{1}: commit: test(LUUL-250): e2e for amount=0
git reset --hard HEAD@{1}          # or: git switch -c rescue 9d281fc
git log --oneline -3
make test-e2e
git push -u origin qa/LUUL-250-more-e2e   # push often: GitHub is your backup
```

Prevent it (platform guardrail)

Push your branch often. Teach the team reflog during onboarding.

Say this in the interview

git reflog records every HEAD movement locally. Lost commits stay recoverable for 30 days by default, so committed work is almost never lost.

Scenario 7: A 2-week big-bang branch

Who	Folder	Script
Bob (Developer) + Omar (Lead)	`$W/bob-dev`	`./sim/scenarios/s07-*.sh`

Problem. feat/big-bang is old, mixes 4 unrelated commits, and one turns ON the unfinished discounts_v2 flag. Omar refuses to review it.

Solve it

```
git fetch
git log --oneline origin/main..feat/big-bang
#  e529d29 test: e2e BLACKFRIDAY
#  5ceaab6 chore: turn on discounts_v2      <- must NOT ship
#  3bdb9a5 docs: API reference
#  c465100 feat: BLACKFRIDAY code

git switch -c feat/LUUL-280-blackfriday origin/main
git cherry-pick c465100 e529d29 && make test && git push -u origin HEAD   # PR 1

git switch -c docs/LUUL-281-api-docs origin/main
git cherry-pick 3bdb9a5 && git push -u origin HEAD                        # PR 2

git push origin --delete feat/big-bang && git branch -D feat/big-bang
```

Prevent it (platform guardrail)

A team agreement (merge at least daily), the PR template checklist, and a branch-age warning in CI. Flags are switched on only in their own PR, approved by lead + QA.

Say this in the interview

I split long branches into small single-purpose PRs by cherry-picking onto fresh branches from main. Each one is easy to review and easy to revert.

Scenario 8: Bug in staging: which commit broke it?

Who	Folder	Script
Sara (QA) + You (Platform)	`$W/sara-qa`	`./sim/scenarios/s08-*.sh`

Problem. GET /price?amount=99.50&code=LUUL10 returns 400. In v1.0.0 it returned 89.55. Six commits have been merged since, and all tests are green.

Solve it

```
cat > /tmp/repro.sh <<'EOF'
#!/usr/bin/env bash
python3 -c "
from app.main import route
s, b = route('/price?amount=99.50&code=LUUL10')
raise SystemExit(0 if s == 200 and b['total'] == 89.55 else 1)"
EOF
chmod +x /tmp/repro.sh
git bisect start HEAD v1.0.0
git bisect run /tmp/repro.sh      # -> refactor: simplify amount parsing
git bisect reset
git switch -c fix/LUUL-290-decimal-amounts
#   app/main.py: int(...) -> float(...)  + add a regression e2e test
make test
git commit -am "fix(LUUL-290): accept decimal amounts"
git push -u origin HEAD
```

Prevent it (platform guardrail)

Every bug becomes a test. QA owns the regression suite in tests/e2e.

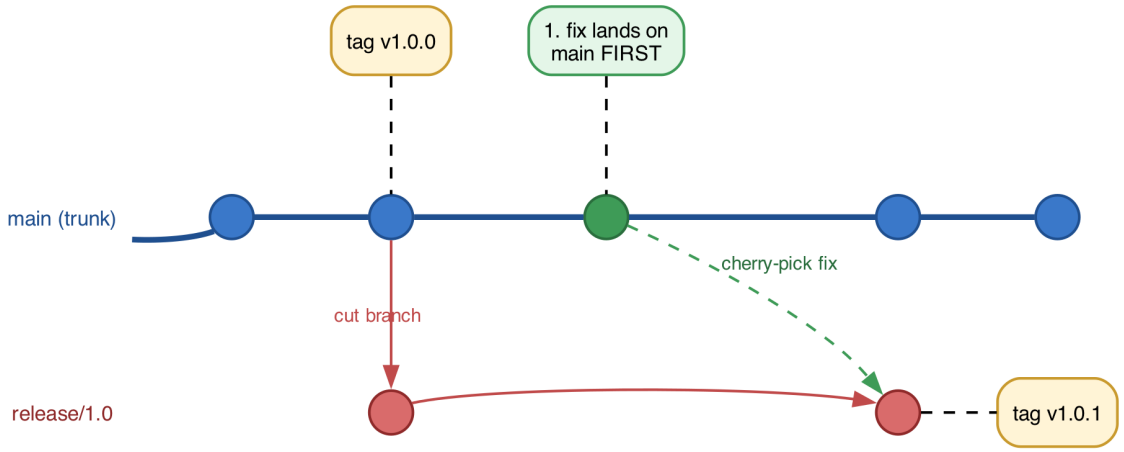
Say this in the interview

QA reproduces the bug as a test, and git bisect finds the guilty commit in a few steps. We fix forward and keep the regression test.

Scenario 9: Hotfix for a version in production

Who	Folder	Script
Omar (Lead) / You (Platform)	`$W/omar-lead`	`./sim/scenarios/s09-*.sh`

Problem. Prod runs v1.1.0 from release/1.1. Codes with spaces (" LUUL10 ") get no discount. Bob's fix is on main, but main also has unfinished work, so main can't ship today.



Fix lands on main first, then it is cherry-picked to the release branch.

Solve it

```
git fetch --tags
git log --oneline v1.1.0..origin/main       # find "fix(LUUL-261): ..."
git switch release/1.1
git cherry-pick -x <fix-sha>                # -x records where it came from
make test && git push
git tag -a v1.1.1 -m "Release 1.1.1" && git push origin v1.1.1
git log --oneline v1.1.0..v1.1.1            # ONLY the fix
```

Prevent it (platform guardrail)

Rule: fix on trunk FIRST, then cherry-pick to the release branch. Release branches are never merged back.

Say this in the interview

We release from trunk with tags. To patch an older release, we cherry-pick the fix from main into the release branch and tag a patch version.

Scenario 10: Commits with the wrong author

Who	Folder	Script
Alice (Developer)	`$W/alice-dev`	`./sim/scenarios/s10-*.sh`

Problem. Alice's new laptop isn't configured, so her 2 commits say root <root@localhost>.

Solve it

```
git log --format='%h %an <%ae> %s' origin/main..HEAD
git config user.name "Alice (Developer)"
git config user.email "alice@luul.lab"
git rebase origin/main --exec 'git commit --amend --no-edit --reset-author'
git log --format='%h %an <%ae> %s' origin/main..HEAD
```

Prevent it (platform guardrail)

An onboarding checklist, and an org rule requiring **signed commits** (SSH or GPG key, shown as "Verified").

Say this in the interview

I fix the identity and rewrite the authors with rebase --exec before pushing. We require signed commits so the audit trail is trustworthy.

10. Practise on real GitHub

The simulator trains your **Git** skills. Real GitHub trains the **platform** part. Follow `docs/GITHUB-MODE.md`:

```
# 1. Create a FREE practice org in the browser (e.g. luul-lab-abdihakim)
# 2. Push the project
git init -b main && git add . && git commit -m "chore: initial luul-orders-service"
gh repo create luul-lab-abdihakim/luul-orders-service --public --source=. --push
# 3. Guardrails
./platform/setup-org.sh luul-lab-abdihakim luul-orders-service     # or Terraform
# 4. First PR: point CODEOWNERS at your org
sed -i '' 's/ORG_NAME/luul-lab-abdihakim/g' .github/CODEOWNERS
```

#	Practise	You learn
G1	Push straight to main	Rulesets reject it (GH013)
G2	Edit ci.yml in a PR	CODEOWNERS requires @platform
G3	CD waits at prod	Environment reviewers = segregation of duties
G4	Flag ON PR	Deploy is not release; lead + QA approve
G5	gh workflow run CD -f sha=	Rollback by redeploying an old commit (40-character ID)
G6	Add/remove team members	Onboarding/offboarding (Terraform PR)
G7	Flaky e2e test	Quarantine with a ticket, never remove the check
G8	Enable merge queue	Fixes Scenario 3 for good
G9	Dependabot PR	Keeping actions and images patched
G10	Break-glass bypass	Emergency path that is audited

What you will see (tested on real GitHub)

1) A brand-new org can be slow: the first push or PR may start no workflow. Wait or push again. **2)** You are in all 4 teams (GitHub adds the creator). **3)** You can't approve your own PR, so alone you merge with `gh pr merge --squash --admin` after checks are green. **4)** Dependabot opens update PRs by itself. **5)** Rollback needs the FULL commit ID: `gh workflow run CD -f sha=$(git rev-parse <short>)`. **6)** Use the script OR Terraform, not both. **7)** Terraform does NOT add you to teams: put your login in members in terraform.tfvars, or nobody can approve prod. **8)** Create protections BEFORE the first push, or the first deploy reaches prod unapproved.

11. Put it on your CV (honestly)

Only write what you have **really done** and can explain line by line. Details: `docs/CV.md`.

When you have rolled it out at Luul Solutions

- Led the move to **trunk-based development** for developers, QA and lead engineers: short-lived branches, squash merges, and **feature flags** to separate deploy from release.
- Designed GitHub **rulesets and CODEOWNERS** with role-based access (devs: write, QA: e2e ownership, leads: maintain + prod approval, platform: admin).
- Built **GitHub Actions** CI/CD: unit, e2e, Docker and secret-scan required checks; SHA-tagged images promoted **dev, staging, prod** with approvals and one-click rollback.
- Managed teams, permissions, rulesets and environments **as code with Terraform**.
- Prevented leaked credentials and broken builds with **push protection, gitleaks, pre-commit hooks and a merge queue**. Wrote runbooks for rollback, hotfix and recovery.

Until then: under Projects

- **luul-orders-service: trunk-based delivery platform (personal project)**. Simulated a 5-person team with role-based GitHub permissions in Terraform, CI/CD with required checks and environment-gated releases, and 10 tested incident runbooks.

> **Say this in the interview**
>
> **S:** long-lived branches, painful merges, big QA batches. **T:** make delivery faster and safer. **A:** trunk-based workflow, rulesets + CODEOWNERS per role, required checks, build once and promote with approvals, feature flags, Terraform, runbooks. **R:** smaller PRs, fewer conflicts, rollback in minutes (use your real numbers).

12. Interview questions and answers (54)

How to practise: cover the answer, say yours **out loud** in under a minute, then compare. Use examples from the scenarios ("In Scenario 3, main went red..."). A questions-only practice sheet is in `docs/Interview-Practice-Sheet.pdf`.

A. The basics

Q1. What is the difference between Git and GitHub?

Git is the version control tool on every laptop: it records commits, branches and history. GitHub hosts Git repositories and adds teamwork features: pull requests, code review, Actions for CI/CD, permissions, rulesets and security scanning.

Q2. Explain trunk-based development in one or two sentences.

Everyone merges small changes into one shared branch, **main**, at least once a day. Branches live hours, not weeks, main is always deployable, and unfinished work is hidden behind feature flags.

Q3. What does "main is always deployable" mean in practice?

Any commit on main could go to production right now. We make that true with required CI checks on every PR, code review, feature flags for unfinished work, and a rule that a red main is fixed or reverted immediately.

Q4. If everyone merges daily, why do you still need pull requests?

The PR is where the safety happens: CI runs the required checks, CODEOWNERS brings in the right reviewer, and there is an audit trail of who approved what. Trunk-based development doesn't mean no review, it means *small and fast* review.

Q5. What is a feature flag?

A switch in config that turns a feature on or off without deploying new code. In our project, **discounts_v2** in config/flags.json is off by default and can be overridden per environment with FLAG_DISCOUNTS_V2=true.

Q6. What is the difference between deploy and release?

Deploy puts code in an environment. **Release** makes a feature visible to users. With flags, we deploy every day with the flag OFF, and release later by turning the flag ON in a small PR, approved by the lead and QA.

Tip: This one sentence shows you understand trunk-based development. Use it often.

B. Trunk-based development in a team

Q7. How do developers, QA and lead engineers work together in your workflow?

Developers make small PRs from main and write unit tests. QA owns the end-to-end tests, which are a required check on every PR and also run against staging, and QA approves when a flag is turned on. Leads review risky code, like pricing, through CODEOWNERS and approve prod deployments. I, as platform, own the pipeline, rules and environments that make this automatic.

Q8. How long should a branch live, and why?

Hours, at most one or two days. The longer a branch lives, the more main moves away from it, so conflicts, review size and risk all grow. Short branches mean small conflicts and fast feedback.

Q9. How big should a pull request be?

Small enough to review properly in about 15 minutes, as a guide a few hundred lines at most, and doing **one** thing. Small PRs are reviewed faster, break less, and are easy to revert.

Q10. If code is merged before it is finished, how does QA test it?

The code is in main with the flag OFF, so users don't see it. QA turns the flag on in dev or staging (FLAG_DISCOUNTS_V2=true) and tests there. When QA is happy, the flag-on PR needs QA's approval. QA also adds e2e tests that run on every PR, so quality is checked continuously, not at the end.

Q11. Why do you use squash merge?

Each PR becomes one clean commit on main, with the PR number and a clear message. History is easy to read, release notes are easy to generate, and if something goes wrong we revert exactly one commit.

Q12. How do you build a feature that takes three weeks?

We still merge every day, in small slices behind a feature flag, or with *branch by abstraction* for big refactors. The feature grows inside main while it is switched off, then it is released with a flag PR.

Q13. How do you keep your branch up to date with main?

git fetch then **git rebase origin/main**, at least every morning and before asking for review. If the branch was already pushed, I push with **--force-with-lease**, which refuses if someone else pushed to my branch.

Q14. How would you move a team from GitFlow to trunk-based development?

Step by step, not in one day. 1) Get CI fast and reliable, with good tests, because trunk-based development depends on it. 2) Protect main with required checks and reviews. 3) Add feature flags. 4) Agree on team rules (daily merges, small PRs) and start with one pilot team. 5) Remove the develop branch, and measure lead time and deploy frequency to show the improvement.

Tip: Interviewers like hearing that you brought people along, not just tools.

C. Roles, permissions and security

Q15. How did you design GitHub permissions for developers, QA, leads and platform?

Teams with least privilege. Developers and QA have **Write**, leads have **Maintain**, platform has **Admin**. Nobody can push to main directly. Responsibility per file comes from CODEOWNERS, and prod deployments need a lead or platform approval through environment reviewers.

Q16. What is CODEOWNERS and how did you use it?

A file that maps paths to teams. When a PR touches a path, GitHub requests that team's review, and with "require code owner review" it must approve. We used it for app/pricing.py (leads), tests/e2e (QA), .github and Dockerfile (platform), and flags.json (leads + QA). The last matching line wins.

Q17. What's the difference between branch protection rules and rulesets?

Both protect branches. Rulesets are the newer system: several rulesets can apply at once and they stack, they have an explicit bypass list, anyone with read access can see them, they can target many branches by pattern, and on paid plans they can apply across the whole organization. Classic branch protection is one rule per branch pattern.

Q18. Who can deploy to production, and how is that enforced?

Only code from main, and only after a lead or platform engineer approves. The prod GitHub environment has required reviewers and a deployment branch policy (main only). The pipeline builds one image per commit and only that image is promoted, so nobody can deploy something that wasn't reviewed and tested.

Q19. Why manage GitHub settings with Terraform instead of clicking in the UI?

Changes are reviewed in a PR, there's history and an audit trail, setups are repeatable across many repos, and drift is visible with terraform plan. Access changes like onboarding become a reviewed PR instead of someone clicking in settings.

Q20. How do you onboard and offboard an engineer?

Onboarding: add them to the right team (a Terraform PR), they set up SSH keys and their git identity, clone, and run make hooks. Offboarding: remove them from the org the same day, revoke their tokens and keys, and rotate any shared secrets they could see.

Q21. How do you keep secrets out of Git?

Layers. A pre-commit hook on laptops, GitHub secret scanning with push protection, a gitleaks job in CI, and .gitignore for .env files. Real secrets live in GitHub Secrets or a vault, and CI uses OIDC to the cloud so there are no long-lived keys to leak.

Q22. What is OIDC in GitHub Actions and why use it?

GitHub gives each workflow run a short-lived signed token. The cloud (AWS, Azure, GCP) trusts it for a specific repo and branch and returns temporary credentials. No cloud keys are stored in GitHub, and nothing long-lived can leak.

D. CI/CD and releases

Q23. Walk me through your pipeline, from commit to production.

A developer opens a PR. CI runs four required checks: unit-tests, e2e-tests, docker-build and secret-scan. CODEOWNERS reviewers approve, and we squash merge. CD then runs the tests again, builds one Docker image tagged with the commit SHA and pushes it to the registry, then deploys dev, then staging (where QA's e2e tests run), then waits for approval and deploys the *same* image to prod.

Q24. Which checks are required on a PR, and who owns each one?

unit-tests are owned by developers. e2e-tests are owned by QA. docker-build and secret-scan are owned by platform. All four are required in the ruleset, and the branch must be up to date with main.

Q25. What does "build once, deploy many" mean?

We build the artifact once per commit and promote the exact same image through every environment. What QA tested in staging is byte-for-byte what goes to prod. Rebuilding per environment could give a different result.

Q26. What is a merge queue and when do you need it?

Before merging, GitHub tests each PR together with the latest main and the PRs ahead of it in the queue. You need it when many PRs merge every day, because two PRs can each be green but break main together.

Tip: Mention Scenario 3: WELCOME5 and WELCOME10.

Q27. How do you version and release?

Semantic versioning (MAJOR.MINOR.PATCH) with tags on main. Pushing a v1.2.0 tag runs the release workflow, which creates a GitHub Release with generated notes. To patch an old version we use a release branch and cherry-pick fixes from main.

Q28. How do you roll back a bad deployment?

The fastest option is to turn the flag off. Otherwise we redeploy the last good image by running CD with the previous commit (gh workflow run CD -f sha=<full SHA>), which also uses that commit's tests and settings, then **git revert** the bad commit on main so trunk is correct again. After that, a blameless postmortem.

Q29. The pipeline is slow. How do you speed it up?

Measure first. Then: cache dependencies and Docker layers, run jobs in parallel, run only what changed (path filters), split long test suites, cancel outdated runs with concurrency, and use bigger or self-hosted runners if needed.

E. What would you do if... (incident questions)

Q30. main is red right after two PRs were merged. What do you do?

Restoring main comes first, because everyone is blocked. I find the commit with **git bisect run make test**, **git revert** it through a fast PR, and tell the authors. Then they fix it properly, and I prevent it with up-to-date checks or a merge queue.

Q31. A developer pushed an AWS key to GitHub. What do you do?

Rotate the key immediately, because it's compromised the moment it's public. Then remove it from the branch history (reset --soft, rm --cached, force-with-lease), or use git filter-repo if it reached main. Check the cloud audit logs for misuse, and add push protection, gitleaks and OIDC so it can't happen again.

Q32. You and another developer conflict on business logic. How do you resolve it?

I don't guess. I talk to the other author and, if it's a business decision, the lead. Then I rebase, resolve the conflict by keeping the agreed result, run the tests and push with --force-with-lease.

Q33. QA finds a regression in staging and nobody knows which commit caused it.

QA writes a test that reproduces the bug. I run **git bisect start HEAD <last-good-tag>** and **git bisect run** with that test to find the commit automatically. We fix forward and keep the test as a regression test.

Q34. Prod has a bug, but main contains unfinished work. How do you hotfix?

If the unfinished work is behind flags, main is still deployable, so we fix on main and ship normally. If prod runs from a release branch, we fix on main first, cherry-pick the fix (-x) into the release branch, and tag a patch version.

Q35. A teammate ran git reset --hard and lost their commits.

git reflog shows every position HEAD had, including the lost commits. Find the commit and use git reset --hard HEAD@{n} or git switch -c rescue <sha>. By default Git keeps lost commits in the reflog for 30 days.

Q36. A developer opens a 2,000-line PR after two weeks of work.

I help them split it, without blaming them: small branches from the latest main, cherry-pick related commits into separate PRs, and any half-finished part behind a flag. Then we look at why it happened (unclear tickets? no flags?) and fix the process.

Q37. A flaky test is blocking every PR.

I don't remove the required check. QA quarantines that one test (skip it with a ticket number) through a PR, the owner fixes it within an agreed time, and we track flaky tests so they don't build up.

F. Hard follow-up questions

Q38. Isn't it risky to merge unfinished code into main?

The risk is lower than with long branches. The code is reviewed and tested, it's switched off by a flag, and it's integrated early, so problems show up while they're small. Big merges after weeks are the risky part.

Q39. What if a lead approves a bad change?

Review is one layer, not the only one. Required tests, staging e2e, gradual flag rollout and fast rollback catch what a reviewer misses. Afterwards we add a test for that case. A blameless postmortem looks at the process, not the person.

Q40. Don't feature flags become technical debt?

They do if nobody owns them. Each flag gets an owner and a ticket to remove it. When a flag has been at 100% for a while, we delete the flag and the old code path. Some teams also fail CI on flags past their expiry date.

Q41. Your merge queue is slow. Now what?

Make CI faster first (caching, parallel jobs). Then tune the queue: group several PRs per build, and only run the slowest tests in the queue rather than on every push.

Q42. Admins can bypass the rules. Isn't that a security hole?

It's a deliberate break-glass path for emergencies. We allow it only for org admins and only when merging a PR, never for direct pushes. Every bypass is logged in the audit log and reviewed afterwards.

Q43. How do you measure whether trunk-based development is working?

The four DORA metrics: deployment frequency, lead time for changes, change failure rate, and time to restore service. Also PR size, branch age and how often main is red.

Q44. Some developers resist the change. How do you convince them?

Listen to the concern first, because it's usually about breaking things. Show them the safety nets (flags, required checks, fast rollback), start with a pilot team, and share results with numbers. People adopt what makes their own day easier.

Tip: A good answer for culture-fit and leadership questions.

G. Working with developers (the developer's point of view)

Q45. How do you, as a DevOps engineer, support developers day to day?

I make the safe path the easy path: one command (**make test**) that runs the same checks locally as in CI, fast pipelines, clear error messages, templates for new services, and docs. I also look at failed pipelines with them and fix the platform when the problem is the platform, not their code.

Q46. A developer says: "It works on my laptop but CI fails." What do you do?

CI is a clean room, so I look for what the laptop has that CI doesn't. Typical causes: a new file that was never committed (**git status** shows ??), a local env var or flag, a different language version, or a package installed only locally. Running the same command as CI (make test) in a fresh clone usually shows it in a minute.

Tip: Tell the story from the developer lab: ModuleNotFoundError for a file that was never git-added.

Q47. QA's tests pass locally but fail in staging. How do you find the cause?

Same code, so I compare **configuration** first: is the right commit deployed (/version)? Are flags and env vars the same (deploy/staging.env)? Then data and dependencies like databases or other services. In our lab the feature flag was OFF in staging, so the endpoint returned 404.

Q48. What is the difference between unit tests and end-to-end tests, and who owns them?

Unit tests call functions directly. They take milliseconds and catch logic bugs, and developers own them. End-to-end tests call the running service over HTTP like a customer and catch integration and config problems. QA owns ours, and they also run against staging. Both are required checks.

Q49. Why should configuration be separate from code?

So the **same image** runs in every environment and only settings change. Then what we tested is what we ship. Config lives in env files, flags or a config service per environment, and secrets in a vault, never baked into the image.

Q50. What is test-driven development and why does it help trunk-based teams?

Write a failing test that describes the behaviour, write the minimum code to pass it, then clean up. It keeps changes small, gives a safety net for merging daily, and makes every PR come with tests.

Q51. How do you decide when a feature flag can be removed, and who removes it?

When the feature has been fully on in prod for a while with no problems, the developer who owns the feature removes the flag and the old code path in a small PR, and platform removes it from the environment settings. Each flag has an owner and a removal ticket from day one.

Q52. A developer wants to skip a failing required check to ship faster. What do you say?

No, but I help. The check protects everyone on trunk. We look at the failure together: a real bug gets fixed, a flaky test gets quarantined with a ticket by its owner, a slow check gets made faster. If a rule really adds no value, we change it with a PR, never with a bypass.

Q53. Which HTTP status codes should an API return, and why does DevOps care?

200 for success, 400 for bad client input, 404 for not found, 500 only when our code crashed. DevOps alerts are built on these numbers. If bad input crashes the app with a 500 instead of returning a 400, on-call gets paged for a client mistake.

Q54. How do health checks and /version help in production?

/health lets Docker, Kubernetes and load balancers know if an instance is alive, so traffic only goes to healthy ones and broken ones get restarted. **/version** shows exactly which commit is running, which is the first question in any incident.

13. Cheat sheet: problem to command

Problem	Command
Committed on main	`git switch -c feat/x` then `git switch main` then `git reset --hard origin/main`
Conflict	`git rebase origin/main`, edit, `git add`, `git rebase --continue`
Push after rebase	`git push --force-with-lease` (your branch only)
main is red	`git bisect run make test` then `git revert <sha>`
Secret pushed	**rotate**, then `git reset --soft origin/main`, `git rm --cached`, force-with-lease
Big file in last commit	`git rm --cached <f>`, .gitignore, `git commit --amend`
Lost commits	`git reflog` then `git reset --hard HEAD@{n}`
Long branch	`git switch -c small origin/main` then `git cherry-pick <shas>`
Who broke it?	`git bisect start HEAD <good>` then `git bisect run <test>`
Patch old release	`git switch release/x`, `git cherry-pick -x <sha>`, tag
Wrong author	`git rebase origin/main --exec 'git commit --amend --no-edit --reset-author'`
Rollback prod	flag OFF, or `gh workflow run CD -f sha=<full-good-sha>`, then revert on main